

CRIPTOGRAFÍA AVANZADA

PRESENTADO POR:
Javier Herranz

ENTIDAD
Universitat Politècnica de
Catalunya (UPC)

MÓDULO
Avanzado

Criptografía Avanzada: un poco de varias cosas

Javier Herranz
Dept. Matemáticas
Universitat Politècnica de Catalunya, Barcelona

Cyber Bootcamp, Univ. Málaga, 16 de julio de 2026



Resumen

- Quién soy yo? Investigador en protocolos criptográficos desde 2001, desde el punto de vista **teórico**.
- Sobre todo en criptografía asimétrica: firmas digitales, cifrado de clave pública, y extensiones de éstos.
- Qué os voy a explicar? Una pincelada de un montón de conceptos criptográficos, sin entrar en mucho detalle (creo) en ninguno de ellos.

Criptografía Simétrica

- Aquella en la que **todos** los usuarios tienen acceso al **mismo** material (claves) criptográfico.
- **Cifrado simétrico:** *one-time pad*, cifrado en *stream*, cifrado de bloque (**AES**), PRG, PRFs...
- **Autenticación simétrica:** códigos de autenticación de mensaje (MACs).
- Seguridad heurística vs. Seguridad (matemáticamente) demostrable.

Funciones de Hash

- Una función $H : \{0, 1\}^* \rightarrow \{0, 1\}^\ell$ tal que:
 - dado $y \in \{0, 1\}^\ell$, es difícil encontrar x tal que $H(x) = y$,
 - es difícil encontrar $x_1 \neq x_2$ tales que $H(x_1) = H(x_2)$.
- Las más usadas (SHA-256, SHA-3) tienen seguridad heurística.
- 1r ejemplo de aplicación, autenticación simétrica **H-MAC**:
 $\tau = H(m, k)$ es un código de autenticación del mensaje m para la clave común k .

Criptografía Asimétrica

- Diferentes usuarios tienen acceso a diferente material criptográfico.
- Esto incluye primitivas como:
 - cifrado de clave pública
 - firma digital
 - pruebas de conocimiento / validez
 - compartición de secretos
 - recuperación privada de información (PIR, OT...)
- Para algunas de ellas, necesitaremos estructuras matemáticas, no solo "caos" heurístico. En particular, **problemas matemáticos difíciles**.

El Problema de la Factorización

- Si $N = p \cdot q$, donde p, q son números primos, cada uno con λ bits de longitud, **no se conoce** ningún algoritmo que **encuentre p, q dado N en tiempo polinómico** en λ .
- El mejor algoritmo actualmente es GNFS (*General Number Field Sieve*): si $\lambda = 1536$, este algoritmo necesita más de 2^{128} operaciones.
- Sistemas criptográficos cuya seguridad se basa en la dificultad de factorizar: RSA, Rabin, Paillier...

El Problema del Logaritmo Discreto (I)

- Si $\mathbb{G} = \langle g \rangle = \{1, g, g^2, \dots, g^{q-1}\}$, es un grupo cíclico con orden primo q (público), el problema del logaritmo discreto consiste en: **dado $y \in \mathbb{G}$, encontrar x tal que $g^x = y$.**
- 1r ejemplo: $\mathbb{G} = \langle g \rangle \subset \mathbb{Z}_p^*$, donde $q \mid p-1$, siendo q y p primos. $\mathbb{G}$ es un grupo cíclico con operación “multiplicación módulo p ”.
- GNFS **también** aplica aquí: para obtener nivel de seguridad 2^{128} , necesitamos $p \geq 2^{3072}$.
- 2o ejemplo: $E = \{(x, y) \in \mathbb{Z}_p^2 \mid y^2 = x^3 + ax + b \bmod p\} \cup \{\mathcal{O}\}$ es una **curva elíptica** sobre $\mathbb{Z}_p$, para un número primo $p > 3$ y constantes $a, b \in \mathbb{Z}_p$, con una operación interna aditiva.

Si tomamos un punto $P \in E$ con orden primo $q \mid \#E$, el subgrupo $\mathbb{G} = \langle P \rangle$ será cíclico con orden q .

- GNFS **no** aplica aquí (lo mejor, Phollard's ρ): podemos obtener nivel de seguridad 2^{128} con $p \geq q \approx 2^{256}$.

El Problema del Logaritmo Discreto (II)

- Problemas relacionados con el problema del Logaritmo Discreto:
 - Problema **Computacional de Diffie-Hellman** (CDH): dados g, g^a, g^b encontrar $g^{a \cdot b}$, siendo a, b aleatorios en $\{0, 1, \dots, q-1\}$.
 - Problema **Decisional de Diffie-Hellman** (DDH): dados g, g^a, g^b, g^c , siendo a, b aleatorios en $\{0, 1, \dots, q-1\}$, distinguir si $c = a \cdot b \bmod q$ o bien si c es aleatorio en $\{0, 1, \dots, q-1\}$.
- Sistemas criptográficos cuya seguridad se basa en la dificultad de resolver el problema del Logaritmo Discreto: Intercambio de claves Diffie-Hellman, cifrado de ElGamal, firma de Schnorr, firma (EC)DSA...

Cifrado de Clave Pública (PKE)

Un sistema de cifrado de clave pública $\mathcal{E} = (\mathcal{E}.\text{KG}, \mathcal{E}.\text{Enc}, \mathcal{E}.\text{Dec})$ consiste en 3 algoritmos PPT:

- **Generación de claves, $\mathcal{E}.\text{KG}$** : la entrada es un parámetro de seguridad k , la salida es una clave pública pk (para encriptar) y una clave secreta sk (para desencriptar). Escribimos $(sk, pk) \leftarrow \mathcal{E}.\text{KG}(1^k)$.
- **Cifrado, $\mathcal{E}.\text{Enc}$** : las entradas son un texto en claro m y una clave pública pk , la salida es un texto cifrado C . Escribimos $C \leftarrow \mathcal{E}.\text{Enc}(m, pk)$.
- **Descifrado, $\mathcal{E}.\text{Dec}$** : las entradas son un texto cifrado C y una clave secreta sk , la salida es un texto en claro $\tilde{m}$. Escribimos $\tilde{m} \leftarrow \mathcal{E}.\text{Dec}(C, sk)$.

Cifrado con RSA

- **Generación de claves**, $(sk, pk) \leftarrow \mathcal{E}.\mathbf{KG}(1^k)$: p y q son primos de $k/2$ bits. Definir $N = p \cdot q$, por tanto $\phi(N) = (p-1)(q-1)$. Escoger una función de hash $H : \mathbb{Z}_N \rightarrow \{0,1\}^\ell$. Tomar $e \in_R \mathbb{Z}_N$ t.q. $\gcd(e, \phi(N)) = 1$, y calcular $d = e^{-1} \bmod \phi(N)$. La clave pública es $pk = (N, e, H)$, la clave secreta es $sk = d$.
- **Cifrado**, $C \leftarrow \mathcal{E}.\mathbf{Enc}(m, pk)$: para encriptar un mensaje $m \in \{0,1\}^\ell$, escoger aleatoriamente $r \in \mathbb{Z}_N$, calcular el texto cifrado $C = (c_1, c_2) = (r^e \bmod N, m \oplus H(r))$.
- **Descifrado**, $\tilde{m} \leftarrow \mathcal{E}.\mathbf{Dec}(C, sk)$: dado un texto cifrado $C = (c_1, c_2)$ y una clave secreta d , devolver el texto en claro

$$\tilde{m} = c_2 \oplus H(c_1^d \bmod N).$$

Cifrado ElGamal

- **Generación de Claves**, $(sk, pk) \leftarrow \mathcal{E}.\text{KG}(1^k)$: $\mathbb{G} = \langle g \rangle$ tiene orden primo q , con k bits. Tomar $x \in_R \mathbb{Z}_q^*$ y calcular $y = g^x$. La clave pública es $pk = (\mathbb{G}, g, y)$, la clave secreta es $sk = x$.
- **Cifrado**, $c \leftarrow \mathcal{E}.\text{Enc}(m, pk)$: para encriptar un mensaje $m \in \mathbb{G}$:
 - escoger $r \in_R \mathbb{Z}_q^*$ aleatoriamente y calcular $c_1 = g^r$,
 - calcular $c_2 = m \cdot y^r$ y definir el texto cifrado $C = (c_1, c_2) \in \mathbb{G}^2$.
- **Descifrado**, $\tilde{m} \leftarrow \mathcal{E}.\text{Dec}(c, sk)$: dado un texto cifrado $C = (c_1, c_2)$ y la clave secreta $sk = x$, devolver

$$\tilde{m} = c_2 \cdot c_1^{-x}$$

Uso Diario de Criptografía Básica: Navegación https://

- HTTPS emplea el protocolo criptográfico **TLS** (*Transport Layer Security*), sucesor de SSL.
- Utiliza un sistema **híbrido** que combina dos tipos de criptografía:
 - Criptografía de Clave Pública: para establecer la conexión inicial (el *handshake*), autenticar al servidor y crear una clave compartida segura k . Actualmente, **RSA**.
 - Criptografía Simétrica: se usa por ejemplo **AES**, con la clave k , para cifrar el resto de la transferencia de datos.

Pausa para Preguntas / Tomar Aire

Esquemas de Firma Digital

Un esquema de firma digital $\Sigma = (\Sigma.KG, \Sigma.Sign, \Sigma.Vfy)$ consta de 3 algoritmos PPT:

- **Generación de Claves, $\Sigma.KG$** : la entrada es un parámetro de seguridad k , las salidas son una clave secreta sk (para firmar) y una clave pública pk (para verificar). Escribimos $(sk, pk) \leftarrow \Sigma.KG(1^k)$.
- **Firma, $\Sigma.Sign$** : las entradas son un mensaje m y una clave secreta sk , la salida es una firma σ . Escribimos $\sigma \leftarrow \Sigma.Sign(m, sk)$.
- **Verificación, $\Sigma.Vfy$** : las entradas son un mensaje m , una firma σ y una clave pública pk , la salida es un bit b : $b = 1$ para 'válida', $b = 0$ para 'falsa'. Escribimos $b \leftarrow \Sigma.Vfy(m, \sigma, pk)$.

Firma RSA

- **Generación de claves**, $(sk, pk) \leftarrow \Sigma.KG(1^k)$: p y q son primos de $k/2$ bits. Definir $N = p \cdot q$, por tanto $\phi(N) = (p-1)(q-1)$. Tomar $e \in_R \mathbb{Z}_N$ t.q. $\gcd(e, \phi(N)) = 1$, y calcular $d = e^{-1} \bmod \phi(N)$. Escoger una función de hash $H : \{0, 1\}^* \rightarrow \{0, 1\}^k$. La clave pública es $pk = (N, e, H)$, la clave secreta es $sk = d$.
- **Firma**, $\sigma \leftarrow \Sigma.Sign(m, sk)$: para firmar un mensaje $m \in \{0, 1\}^*$, calcular $\sigma = H(m)^d \bmod N$.
- **Verificación**, $b \leftarrow \Sigma.Vfy(m, \sigma, pk)$: dado un mensaje m , una firma σ y la clave $pk = (N, e, H)$, devolver $b = 1$ si y sólo si

$$\sigma^e = H(m) \bmod N.$$

Firma de Schnorr

- **Generación de claves**, $(sk, pk) \leftarrow \Sigma.KG(1^k)$: $\mathbb{G} = \langle g \rangle$ tiene orden primo q , con k bits. Tomar $x \in_R \mathbb{Z}_q^*$ y calcular $y = g^x$. Escoger una función de hash $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q$. La clave pública es $pk = (\mathbb{G}, g, y, H)$, la clave secreta es $sk = x$.
- **Firma**, $\sigma \leftarrow \Sigma.Sign(m, sk)$: para firmar un mensaje $m \in \{0, 1\}^*$, escoger $r \in_R \mathbb{Z}_q$ aleatoriamente, calcular $R = g^r$ y $s = r + x \cdot H(m, R) \bmod q$. La firma es $\sigma = (R, s)$.
- **Verificación**, $b \leftarrow \Sigma.Vfy(m, \sigma, pk)$: dado un mensaje m , una firma $\sigma = (R, s)$ y la clave $pk = (\mathbb{G}, g, y, H)$, devolver $b = 1$ si y sólo si

$$g^s = R \cdot y^{H(m, R)}$$

Emparejamientos Bilineales (*Pairings*)

Sean $\mathbb{G}_1 = \langle g_1 \rangle$ y $\mathbb{G}_2 = \langle g_2 \rangle$ dos grupos cíclicos con el mismo orden primo q .

- $\mathbb{G}_1, \mathbb{G}_2$ admiten un **emparejamiento bilineal** (*bilinear pairing*) cuando existe otro grupo cíclico $\mathbb{G}_T$ de orden q y una aplicación $e : \mathbb{G}_1 \times \mathbb{G}_2 \longrightarrow \mathbb{G}_T$ tal que
 - 1 $e(\cdot, \cdot)$ puede ser evaluado en tiempo polinómico (en $\log(q)$),
 - 2 $\mathbb{G}_T = \langle e(g_1, g_2) \rangle$,
 - 3 $\forall a, b \in \mathbb{Z}_q$, se cumple $e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}$.

Si $\mathbb{G}_2 = \mathbb{G}_1$, se dice que el emparejamiento es **simétrico**.

Pairings y Criptografía

- Las primeras relaciones entre emparejamientos bilineales y criptografía fueron “**negativas**”:
 - ① si $\mathbb{G}$ admite un emparejamiento simétrico, el problema DDH pasa a ser **fácil** en $\mathbb{G}$,
 - ② si $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ es emparejamiento bilineal, el problema del Logaritmo Discreto en $\mathbb{G}_1$ o en $\mathbb{G}_2$ es al menos tan fácil como en $\mathbb{G}_T$ (impacto para criptografía basada en curvas elípticas).
- **Primeros usos positivos** de los emparejamientos bilineales:
 - Intercambio de claves entre 3 usuarios, no-interactivo (Joux, 2000).
 - Esquema de firma de Boneh-Lynn-Shacham (BLS, 2001)
 - Esquema de cifrado basado en identidades (Boneh-Franklin, 2001).

Firma BLS

- **Generación de Claves**, $(sk, pk) \leftarrow \Sigma.\mathbf{KG}(1^k)$: generar $\mathbb{G}_1 = \langle g_1 \rangle$ y $\mathbb{G}_2 = \langle g_2 \rangle$ con orden primo q , de longitud k bits, que admitan un emparejamiento bilineal $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$.
Tomar $x \in_R \mathbb{Z}_q^*$ y calcular $y = (g_2)^x \in \mathbb{G}_2$. Escoger una función de hash $H : \{0, 1\}^* \rightarrow \mathbb{G}_1$.
La clave pública es $pk = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, y, H)$,
la clave secreta es $sk = x$.
- **Firma**, $\sigma \leftarrow \Sigma.\mathbf{Sign}(m, sk)$: la firma (determinista) de un mensaje $m \in \{0, 1\}^*$ es $\sigma = H(m)^x \in \mathbb{G}_1$.
- **Verificación**, $b \leftarrow \Sigma.\mathbf{Vfy}(m, \sigma, pk)$: dada una pareja (m, σ) de mensaje y firma, y la clave pública pk , devolver $b = 1$ si y sólo si

$$e(\sigma, g_2) = e(H(m), y)$$

Comparación entre Diferentes Firmas

Para un nivel de seguridad de 128 bits:

Esquema de firma	$ pk $	$ sk $	$ \sigma $
RSA	3072	3072	3072
Schnorr (subgrupos de $\mathbb{Z}_p$)	3072	200	400
Schnorr (curva elíptica) o ECDSA	300	256	500
BLS	512	256	256

La Amenaza Cuántica

- En los años 90, aparecieron dos algoritmos cuánticos:
 - El algoritmo de búsqueda de **Grover** (para secuencias no ordenadas de N elementos), que tiene coste $\mathcal{O}(\sqrt{N})$.
 - Los algoritmos de **Shor**, que resuelven el problema de factorizar N y el problema del logaritmo discreto en **tiempo polinómico** (en el parámetro de seguridad).

Éstos últimos implican que esquemas como RSA, ElGamal, Schnorr, DSA, Paillier, BLS... **serán inseguros en un mundo con ordenadores cuánticos** potentes.

Criptografía Resistente a Ordenadores Cuánticos

- La criptografía con seguridad heurística (AES, funciones hash SHA...) sólo se ve afectada por el algoritmo de Grover.

Solución: duplicar todos los tamaños. Aplica a criptografía **simétrica** y algunas soluciones asimétricas **basadas en funciones de hash**.

- Otros problemas matemáticos **de momento resisten** ataques por ordenadores cuánticos:
 - problemas de teoría de códigos,
 - problemas con polinomios multivariados,
 - problemas relacionados con isogéneas entre curvas elípticas
 - problemas relacionados con retículos euclídeos (***lattices***).
- **NIST** en proceso de estandarizar esquema de firma digital y cifrado de clave pública con seguridad post-cuántica.

Ejemplo: Firmas Basadas en Funciones Hash (I)

El esquema de Lamport (1979) para firmar **un único mensaje**:

- 1 Se escoge una función de hash segura $H : \{0, 1\}^{\ell_1} \rightarrow \{0, 1\}^{\ell_2}$.
- 2 Se escogen $2\ell_2$ valores aleatorios $x_{i,j} \in_R \{0, 1\}^{\ell_1}$, para $i = 1, 2, \dots, \ell_2, j = 0, 1$, que formarán la clave **secreta**.
- 3 Se calculan los valores $y_{i,j} = H(x_{i,j})$, que formarán la clave **pública**.
- 4 Para firmar un mensaje $m \in \{0, 1\}^{\ell_1}$, primero se calculan los bits

$$H(m) = (m_1, m_2, \dots, m_{\ell_2}) \in \{0, 1\}^{\ell_2}$$

La firma es $\sigma = (x_{1,m_1}, \dots, x_{\ell_2,m_{\ell_2}})$.

- 5 Dado un mensaje $m \in \{0, 1\}^{\ell_1}$ y una firma $\sigma = (\sigma_1, \dots, \sigma_{\ell_2})$, primero se calculan los bits

$$H(m) = (m_1, m_2, \dots, m_{\ell_2}) \in \{0, 1\}^{\ell_2}$$

La firma se acepta como válida si y sólo si $H(\sigma_i) = y_{i,m_i}$, para todo $i = 1, 2, \dots, \ell_2$.

Ejemplo: Firmas Basadas en Funciones Hash (II)

Se puede combinar un esquema de firma de un sólo uso (o de pocos usos) con los **Árboles de Merkle**, para conseguir esquemas de firma de múltiples usos.

- El valor en cada **hoja** del árbol binario es una clave pública diferente del esquema de un uso.
- El valor de un nodo superior es el **Hash** de la concatenación de sus dos hijos.
- La clave pública total es el valor en el **nodo raíz**.
La clave secreta es (una semilla que genere) el conjunto de claves secretas de un solo uso.
- La firma del mensaje i -ésimo m_i es $\sigma_i = \Sigma.\text{Sign}(m_i, sk_i)$ junto con pk_i y los valores de los nodos hermanos del camino hasta la raíz.

Criptografía Distribuida: Observaciones

- Ingrediente principal: esquemas de **compartición de secretos**.

secreto $s \longrightarrow$ fragmentos $s_1, s_2, \dots, s_n$

de manera que (sólo) si $A \subset \mathcal{P}$ es autorizado:

$$s = \sum_{P_i \in A} \lambda_i^A \cdot s_i$$

- Más **fácil** si el secreto s pertenece a un **cuerpo finito conocido públicamente**.
- Más **fácil** si la tarea secreta es **determinista**, sin aleatoriedad (compartida).

MultiFirmas

n personas que firman el mismo mensaje m , cada uno usando su clave secreta sk_i . El objetivo es que la multifirma resultante σ ocupe menos que tener n firmas independientes.

- Para poder verificar la multifirma σ , debe publicarse la lista con los firmantes que intervienen en el cálculo de la firma, y sus claves públicas.
- La firma resultante σ , junto con el mensaje m y todas las claves públicas de todos los firmantes $pk_1, \dots, pk_n$ sirven para verificar que **todos ellos** han cooperado para firmar m .

Ejemplo: Multifirmas BLS (básicas)

Recordemos: una firma σ es válida para el mensaje m y la clave pública $y = g^x$ sii $\sigma = H(m)^x$, es decir:

$$e(\sigma, g) = e(H(m), y)$$

- Cada usuario i , con clave secreta x_i y clave pública $y_i = g^{x_i}$, firma m como $\sigma_i = H(m)^{x_i}$ y publica σ_i .
- La multifirma es $\sigma = \prod_{i=1}^n \sigma_i = \dots = H(m)^{\sum_{i=1}^n x_i}$.
- Una multifirma es aceptada si cumple

$$e(\sigma, g) = e(H(m), \prod_{i=1}^n y_i)$$

Firmas Agregadas

- Misma idea que en las multifirmas, pero para el caso en que los firmantes firmen **mensajes distintos**.
- El algoritmo de verificación tomará como entradas la lista de parejas $\{(pk_i, m_i)\}_{1 \leq i \leq n}$ y la firma agregada σ .
- Objetivo 1: una firma agregada σ válida sólo puede calcularse a partir de firmas σ_i válidas.
- Objetivo 2: el **tamaño** de σ y el **coste de verificar** su validez deben ser (mucho) **menores** que el de enviar y verificar las n firmas individuales.

Comparación

Tipo de firma	MultiFirma	Agregada
Anonimato de firmantes frente a verificador	no	no
Firmar mensajes diferentes	no	sí
Generación de claves no interactiva	sí	sí
Eficiente con Log.Discreto	sí	no
Eficiente con RSA	no	no
Eficiente con <i>Pairings</i>	sí	sí

Casos de Uso

- Gnosis Safe: <https://safe.global/>
- Puentes *cross-chain*:
<https://www.gate.com/es/learn/articles/what-are-cross-chain-bridges-a-deep-dive-into-the-key-infra-10218> dónde se usan las multifirmas o las firmas agregadas, preguntar a Claude...
- Validación de bloques en Blockchain, **Beacon Chain**:
<https://www.ledger.com/academy/ethereum-beacon-chain-explained>
https://eth2book.info/latest/part2/building_blocks/signatures/
- Otros usos de firmas agregadas: redes de sensores IoT, redes de roaming...

Pausa para Preguntas / Café

Pruebas Interactivas (de Conocimiento / Validez)

- Dados unos elementos / parámetros comunes pms y un elemento público y , un participante / **probador** P quiere demostrar a un **verificador** V que conoce alguna información ω sobre y .
- Para hacerlo, podrán intercambiarse mensajes. Al final, el verificador aceptará o rechazará la prueba.
- Normalmente ω es una información privilegiada, (muy) difícil de calcular a partir sólo de y .
- Ejemplos:
 - $pms = \mathbb{G} = \langle g \rangle$, y w es tal que $g^w = y$ (Prueba de Conocimiento del Logaritmo Discreto).
 - $pms =$ función de hash H , y w es tal que $H(w)$ empieza por y ceros.
 - $y = (y_1, y_2)$, donde $y_1 = \text{PKE.Enc}(w)$ y $w \geq y_2$.

Conocimiento Nulo (*Zero-Knowledge*)

- En muchos casos de uso, P quiere mantener la información privilegiada w en secreto:

La ejecución del protocolo interactivo no tiene que proporcionar a V más información sobre w de la que se deduce de lo que se está demostrando.

- Por ejemplo, convencer a alguien que conoces w tal que $y = g^w$ es un **protocolo de Identificación** para la clave pública $pk = y$. Si el protocolo ofrece *Zero-Knowledge*, podrá ejecutarse muchas veces.
- En otros escenarios no se requiere *Zero-Knowledge*, y lo que se busca es que verificar la prueba sea **más eficiente** para V que hacer otro cálculo: $pms = (f, x)$, P publica y , $\omega = \emptyset$ y V quiere convencerse que $y = f(x)$.

Ejemplo: Conocimiento de un Logaritmo Discreto

$$\text{pms} = (q, \mathbb{G} = \langle g \rangle)$$

$$y \in \mathbb{G}$$

$$\omega \text{ es tal que } y = g^\omega$$

- 1 P elige $r \in_R \mathbb{Z}_q$ aleatoriamente, envía $R = g^r$ a V .
- 2 V elige (un reto) $h \in_R \mathbb{Z}_q$ y lo envía a P .
- 3 P calcula $s = r + h \cdot \omega \bmod q$ y envía s a V .

V acepta la prueba sii $g^s = R \cdot y^h$.

Pruebas No Interactivas: el Paradigma Fiat-Shamir

- En 1986, **Fiat y Shamir** propusieron una manera de transformar algunas pruebas de conocimiento interactivas en pruebas no interactivas.
- Para que la transformación funcione, todos los valores que V envía a P deben ser **aleatorios e uniformes**.
- La transformación consiste en remplazar cada uno de esos valores por H (mensajes intercambiados previamente), para una función de hash H segura.

Fiat-Shamir aplicado al ejemplo anterior

$$\text{pms} = (q, \mathbb{G} = \langle g \rangle, H : \{0, 1\}^* \rightarrow \mathbb{Z}_q)$$

$$y \in \mathbb{G}$$

ω es tal que $y = g^\omega$

- 1 P elige $r \in_R \mathbb{Z}_q$ aleatoriamente, calcula $R = g^r$.
- 2 P calcula $h = H(R)$.
- 3 P calcula $s = r + h \cdot \omega \bmod q$.

P envía (R, s) a V .

V acepta la prueba sii $g^s = R \cdot y^h$, siendo $h = H(R)$.

Es el **protocolo de Identificación** no interactiva (Schnorr, 1991).

Recuperando la Firma de Schnorr

De ese protocolo no interactivo de identificación a una firma digital falta sólo una cosa... el mensaje m .

Hay sitio para m entre las entradas de la función de hash !!

$$pk = (q, \mathbb{G} = \langle g \rangle, y, H : \{0, 1\}^* \rightarrow \mathbb{Z}_q)$$

$$sk = \omega \text{ tal que } y = g^\omega$$

Para firmar un mensaje m :

- 1 elegir $r \in_R \mathbb{Z}_q$ aleatoriamente, calcular $R = g^r$, luego calcular $s = r + \omega \cdot H(\textcolor{red}{m}, R) \bmod q$.
- 2 La firma es $\sigma = (R, s)$.

Una firma $\sigma = (R, s)$ es válida para el mensaje m sii

$$g^s = R \cdot y^{H(m, R)}$$

Un Lenguaje Más Complejo: las Pruebas *OR*

- P conoce **uno** de los logaritmos discretos de una lista $y_1, y_2, \dots, y_n \in \mathbb{G} = \langle g \rangle$ con respecto a la base g .
- Es decir, $\text{pms} = (q, \mathbb{G}, g)$, $y = (y_1, \dots, y_n)$ y ω satisface

$$y_1 = g^\omega \text{ OR } y_2 = g^\omega \text{ OR } \dots \text{ OR } y_n = g^\omega$$

- Ahora el tamaño del lenguaje o circuito satisfecho por ω es $\mathcal{O}(n)$.
- Si $y_i = pk_i$, una prueba ZK de este lenguaje da pie a un sistema de **Identificación Anónima dentro de un grupo** (elegido *ad-hoc*).

Firmas de Anillo / Grupo (Anonimato)

- Aplicando Fiat-Shamir y añadiendo un mensaje m , se obtiene una **firma de anillo**:

La firma resultante convence de que un miembro del anillo $\{pk_1, \dots, pk_n\}$ ha firmado m , **sin revelar nada sobre la identidad del firmante**.

- En las **firmas de grupo**, también anonimato del firmante, pero:
 - las claves (sk_i, pk_i) las crea una entidad máster de grupo,
 - la entidad máster puede romper el anonimato de una firma,
 - el firmante no puede elegir el grupo/anillo de posibles firmantes.
- Caso de Uso: criptomoneda **Monero**.

Avances en Pruebas (ZK) de Conocimiento

- Usando las técnicas clásicas (90s), una prueba para un lenguaje OR en el escenario de Log.Discreto tenía longitud y coste de verificación $\mathcal{O}(n)$.
- Las **pruebas de rango**, en las que se demuestra que un valor cifrado/comprometido x pertenece al intervalo $[a, b]$, son un caso particular de OR proof, con $n = b - a$ casos posibles.
- Situación muy habitual en **cripto-monedas**, para demostrar que una cuenta tiene fondos suficientes para realizar una compra.
- En los últimos 15 años se han producido diversos avances en este área de pruebas ZK, sobre todo para **mejorar la longitud y tiempo de verificación** de estas pruebas.

Pruebas zk-SNARKs (desde 2013)

- Demostrar conocimiento de ω tal que $f(\omega) = y$ con tamaño de la prueba (y coste de verificación) **constante**, independiente del tamaño de f .
- Requiere una inicialización de confianza (con valores tóxicos que deben ser eliminados), específica para cada función f . Es la *Trusted Setup Ceremony*.
- Se utilizan *emparejamientos bilineales* y la seguridad se reduce a una hipótesis un poco extraña (*non-falsifiable*).
En particular, **no se obtiene seguridad post-cuántica**.

Bullet-Proofs (desde 2018)

- Basados en el escenario clásico de Log. Discreto, **sin emparejamientos bilineales**.
- Ahora el tamaño de las pruebas y el coste de verificación **no son constantes**, sino $\mathcal{O}(\log(|f|))$.
- Como aspecto positivo, no se requieren ceremonias de inicialización de confianza.
- **No** ofrece seguridad post-cuántica.

Pruebas zk-STARKs (desde 2017)

- Mismas propiedades de eficiencia que Bullet-Proofs: $\mathcal{O}(\log(|f|))$.
- Tampoco se requieren ceremonias de inicialización de confianza.
- Basados en **árboles de Merkel** (funciones de hash) para comprometerse a ω , además de pruebas relacionadas con **polinomios** (seguridad incondicional), e.g. FRI.
- Por tanto, **sí** ofrecen seguridad post-cuántica.

Alternativa PQ: ZK a partir de *MPC-in-the-Head*

- Idea inicial en un artículo de 2007.
- El probador **P** ejecuta internamente un protocolo MPC entre n usuarios ficticios, publica $H(\text{view}_i)$ para cada uno de ellos, y finalmente revela algunas view_j 's elegidas por **V**.
- Menos eficiente que STARK, pero muy versátil → nuevos sistemas de **firma con seguridad post-cuántica** (e.g. **Picnic**).

Aplicación: MultiFirmas basadas en Funciones Hash

- Sea Σ un esquema de firma digital basado en funciones de hash (XMSS, SPHINCS...).
- Cada firmante P_i calcular su firma σ_i del mensaje común m , válida para su clave pública pk_i .
- Un agregador calcula la MultiFirma: una **prueba (STARK)** del conocimiento de $\omega = (\sigma_1, \dots, \sigma_n)$ tal que $\Sigma.\text{Vfy}(m, \sigma_i, pk_i) = 1$ para todo $i = 1, \dots, n$.
- No hace falta siquiera que la prueba sea de conocimiento nulo.
- La firma tiene tamaño $\mathcal{O}(\log(n))$, **peor** que las soluciones $\mathcal{O}(1)$ basadas en BLS, Schnorr o *lattices*.
- A cambio, la seguridad se basa 100% en **herramientas simétricas** (funciones de hash), en particular post-cuántica.

Casos de Uso de Pruebas de Conocimiento

- Escalabilidad de blockchains (**ZK-Rollups**): validar que las transacciones fuera de la cadena principal (zkSync, StarkNet, Polygon zkEVM, Scroll...) se realizan correctamente.
- Privacidad en transacciones financieras, como **Zcash**, para demostrar que no hay doble gasto, que los balances cuadran... sin revelar nada más.
- **ZKML**, Zero-Knowledge Machine Learning, para demostrar que un modelo ML se ha entrenado o ejecutado de manera correcta.
- Identidad Digital / Credenciales Anónimas: demostrar tu nacionalidad, tu mayoría de edad... sin revelar nada más. Proyecto **eIDAS** en Europa.

Pausa para Preguntas / Tomar Aire

Cifrado Homomórfico

- Permite **operar sobre datos cifrados**.

- **Aditivamente** homomórfico:

$\text{PKE.Enc}(m_1, pk) \oplus \text{PKE.Enc}(m_2, pk)$ es un cifrado de $m_1 + m_2$.

Ejemplos: Paillier, Joye-Libert, algunos basados en *lattices*.

- **Multiplicativamente** homomórfico:

$\text{PKE.Enc}(m_1, pk) \otimes \text{PKE.Enc}(m_2, pk)$ es un cifrado de $m_1 \cdot m_2$.

Ejemplo de ElGamal:

$$\text{PKE.Enc}(m_1, pk) = (g^{r_1}, m_1 \cdot y^{r_1})$$

$$\text{PKE.Enc}(m_2, pk) = (g^{r_2}, m_2 \cdot y^{r_2})$$

$\otimes$ es multiplicación componente a componente:

$$\text{PKE.Enc}(m_1, pk) \otimes \text{PKE.Enc}(m_2, pk) = (g^{r_1+r_2}, m_1 \cdot m_2 \cdot y^{r_1+r_2}).$$

Cifrado Completamente Homomórfico (FHE)

- Si un sistema de cifrado es homomórfico **tanto para la suma como para la multiplicación**, es completamente homomórfico (*Fully Homomorphic Encryption*, FHE).
- Primer candidato en 2009 (**C. Gentry**), desde entonces un área muy activa. Todos los candidatos basados en *lattices*.
- Estos sistemas de cifrado funcionan con **ruido**; operar entre cifrados va aumentando el ruido, lo que hace necesaria la operación de **bootstrapping**, muy costosa.
- En teoría, estos sistemas permiten **delegar cualquier computación** sobre datos cifrados.

Casos de Uso

- Computación en la nube de funciones sobre datos sensibles (médicos).
- ML cifrado, MLaaS: entrenar o ejecutar modelos IA sobre datos cifrados (empresas como Zama, Duality o Microsoft (SEAL)).
- Contratos inteligentes privados en Blockchain, por ejemplo proyectos como Fhenix o Inco Network.

Caso de Uso Global: Voto Electrónico (I)

- Hay una clave pública pk de cifrado para la votación.
- Cada votante P_i tiene unas claves (pk_i, sk_i) , la lista de claves públicas del **censo de votantes** es conocida.
- El votante P_i encripta su opción de voto $c_i = \text{PKE.Enc}(m_i, pk)$ y firma el cifrado c_i con su clave secreta, $\sigma_i = \Sigma.\text{Sign}(c_i, sk_i)$. Envía (c_i, σ_i, pk_i) a la urna inicial, no anónima.
- Para cada entrada (c_i, σ_i, pk_i) , se verifica que pk_i pertenece al censo, y que $\Sigma.\text{Vfy}(c_i, \sigma_i, pk_i) = 1$. Los votos cifrados **válidos** c_i van a la siguiente urna.

Voto Electrónico (II)

- Dos posibilidades para **anonimizar** la urna $C = \{c_1, \dots, c_\ell\}$:
 - Si la elección lo admite (referéndums...), aplicar **tally homomórfico**: usar propiedad homomórfica de PKE para generar un único texto cifrado $c = \bigoplus_{1 \leq i \leq \ell} c_i$ que encripte por ejemplo $\sum_{1 \leq i \leq \ell} m_i$.
 - Ronda de **mezclado** / *shuffling* verificable: re-aleatorizar y permutar todos los cifrados de la urna C (varias veces).
- Para terminar, la(s) autoridad(es) de la elección, que dispone de *sk* (o fragmentos de *sk*), descifran c (tally) o todos los c_i 's (shuffling) y publican el resultado de la votación.

Voto Electrónico (III)

Muchas amenazas / propiedades a tener en cuenta...

- El descifrado final tiene que ser verificable: **pruebas ZK**.
- En el caso de *tally homomórfico*, cada votante tiene que demostrar que c_i encripta un voto permitido: **pruebas ZK**.
- En el caso de *shuffling*, cada nodo tiene que demostrar que ha realizado una permutación y re-aleatorización correcta: **pruebas ZK**.
- ¿Cómo evitar la **compra/venta de votos**?
La solución presentada permite a P_i demostrar a un comprador que c_i encripta m_i (por ejemplo mostrando la aleatoriedad usada al encriptar).
- Y la amenaza cuántica? Firma? Cifrado?

CRIPTOGRAFÍA AVANZADA

PRESENTADO POR:
Javier Herranz

ENTIDAD
Universitat Politècnica de
Catalunya (UPC)

MÓDULO
Avanzado